

Ramanujan Laboratory Pro: Research Manual

Contents

1 1. Orientation and reliability

Purpose of the laboratory

Ramanujan Laboratory Pro is a research assistant for eta products, formal q-series, dissections, arithmetic progressions, modular-form certificates, and exploratory computation. The program is designed around a strict separation between three kinds of output.

Exact symbolic output is obtained by algebraic substitution from a verified identity. The app records every identity, scaling, power, and recombination. **Certified modular output** is obtained only after the eta-quotient hypotheses and the Sturm bound have been checked. **Computational evidence** is based on finitely many coefficients and is displayed as a conjectural or experimental result.

First example

Enter

$$F(q) = \frac{f_1^2}{f_2} - \frac{f_1^6}{f_2^3}$$

in the Automatic p-Dissection Solver and select p=3. The app searches its verified identity library term by term, constructs the three residue components, checks the resulting equality by expanding both sides, and generates a proof trace in readable text and LaTeX.

Navigation

The left menu is divided into exact algebra, modular certification, exploratory tools, external services, and documentation. Start with Home or the Automatic p-Dissection Solver. Use the Composite & Residue Lab when you need iterated progressions such as $a(8n+7)$. Use the Sturm Certificate only when both sides are eta quotients of the same modular type.

Reliability rule

A long successful coefficient check is not automatically a proof. It becomes a Sturm proof only when the app verifies the relevant modular-form hypotheses and checks through the exact bound. The manual repeats this distinction because it is the most important methodological safeguard in computational partition theory.

Operational checklist

Before trusting an output, record the formula exactly, the selected truncation degree, the dissection base, and the application version shown in the sidebar. Re-run the calculation after clearing the cache when a formula has been edited substantially. For an exact result, open the derivation tab and confirm that every stored identity is appropriate for the scale used. For a certified result, inspect the modularity table rather than relying only on the green status message. For an experimental result,

export both the coefficient list and the conjectured formula so the computation can be reproduced independently.

The app is intended to shorten routine algebra, not to replace mathematical judgment. A clean workflow is: discovery, independent verification, theorem selection, proof construction, and final exposition. The same formula may pass through several modules during these stages.

2 2. Input notation and q-series engine

Supported notation

The fundamental symbol is

$$f_m = (q^m; q^m)_\infty.$$

Write f_1 , f_{24} , f_1^3 , products, quotients, sums, and differences. Standard examples are

$$\frac{1}{f_1 f_3}, \quad \frac{f_2^5}{f_1^2 f_4}, \quad f_1^2 - f_2^2.$$

The parser also supports selected theta functions such as $\varphi(q^2)$, $\psi(q^3)$, Ramanujan's general theta function, and the Rogers-Ramanujan auxiliary product used in the stored 5-dissections.

Exact arithmetic

The coefficient engine uses rational arithmetic. Division is allowed only for formal power series with a nonzero constant term. Therefore expressions such as $1/f_1$ are legitimate, while division by a series beginning with q is rejected.

Truncation

A requested degree N means that calculations are performed modulo $q^{(N+1)}$. Products and inverses are truncated consistently. When an operator needs $a(pn)$, the engine automatically computes far enough to obtain the requested output degree.

Safe evaluation

The LaTeX-like input is converted to a restricted expression tree. Arbitrary Python attributes, imports, and function calls are not allowed. This is important on a public webpage.

Example

For $1/f_1$, request degree 10. The app returns the partition numbers

$$1, 1, 2, 3, 5, 7, 11, 15, 22, 30, 42.$$

Use this simple test whenever you deploy a new version. It checks parsing, inversion, multiplication, and coefficient display simultaneously.

Parsing examples and diagnostics

Implicit multiplication is supported in standard eta-product notation, but unambiguous input is always better. Prefer `f_1^2*f_3` when debugging a difficult expression. Fractions should use complete braces, for example `\\frac{f_2^5}{f_1^2 f_4}`. If the parser reports an unknown symbol, remove decorative LaTeX commands and test the smallest subexpression first.

A useful diagnostic sequence is to compare the first coefficients with a trusted computer algebra system. Test `f_1`, `1/f_1`, and the proposed expression separately. When a denominator is present, check its constant coefficient. The formal inverse exists only when that coefficient is nonzero. Exact rational coefficients are retained throughout, so a nonintegral output is not silently rounded. External tools such as OEIS require integral coefficients and will stop with an explanatory message otherwise.

3 3. Dissections and identity paths

Meaning of a p-dissection

If $F(q) = \sum a(n)q^n$, its p-dissection is

$$F(q) = F_0(q^p) + qF_1(q^p) + \cdots + q^{p-1}F_{p-1}(q^p),$$

where

$$F_r(q) = \sum_{n \geq 0} a(pn + r)q^n.$$

The exact solver uses stored identities for $p=2,3,5$. Bases 7 and 11 are available through coefficient extraction even when no finite eta-product identity is stored.

Step-by-step path

For each term, the app identifies a matching left side, records a possible scaling $q \rightarrow q^m$, raises the identity to the necessary power, multiplies by factors already depending on q^p , and then recombines all terms. Every step appears in the Derivation tab and in the LaTeX export.

Example

The identity

$$\frac{f_1^2}{f_2} = \frac{f_9^2}{f_{18}} - 2q \frac{f_3 f_{18}^2}{f_6 f_9}$$

is a 3-dissection. The first term contains only powers divisible by 3 and the second lies in residue 1. Thus the residue-2 component is zero.

Identity explanation

In the verified library, press Explain identity. The dialog gives the role of the identity, the stored provenance note, and a fresh coefficient audit. For publication, replace generic provenance notes with the exact bibliographic equation number in your personal identity library.

How to add a new dissection safely

First enter the proposed identity in the Personal Identity Library and verify it to a reasonably high degree. Add a precise source: author, paper or book, equation number, and any notation conversion. Next test one or two scaled versions $q \rightarrow q^m$. Finally compare the extracted residue components directly. Only after these checks should the identity be promoted into the built-in library.

For $p=7$ or 11, the app always provides coefficient components. A finite closed form may involve auxiliary theta functions rather than eta products alone. In that case, use the coefficient output

to determine the shape, add the required special function to the parser, and store the exact source identity. Avoid forcing a large component into an artificial eta quotient merely because a short coefficient fit exists.

4 4. Composite expressions and deep extraction

Sums and differences

The Composite & Residue Lab accepts finite linear combinations of eta monomials. This is essential for functions constructed as differences, where cancellation may create stronger congruences than either summand has separately.

Direct extraction

For $p=2$, the components are $a(2n)$ and $a(2n+1)$. For $p=3$, they are $a(3n)$, $a(3n+1)$, and $a(3n+2)$. Select the residue and inspect both the symbolic component and its coefficient check.

Deep extraction

To obtain $a(8n+7)$, write 7 in base 2:

$$7 = 1 + 2 + 4 = (111)_2.$$

The app successively selects the odd component three times:

$$a(2n+1) \longrightarrow a(4n+3) \longrightarrow a(8n+7).$$

At every level it records the current generating function, the identity substitutions, and the selected digit.

Product recognition

When no closed symbolic expression is found, the app computes the component coefficients and applies logarithmic derivatives and Möbius inversion. If the Euler exponents stabilize to a simple finite pattern, it proposes an f-product. This is a recognition step, not a proof, until the proposed identity is independently established.

Practical advice

Use a moderate branch limit first. Large powers of multi-term dissections can grow exponentially. If the symbolic search becomes too large, use coefficient extraction to identify the likely simple component and then add a suitable identity to the personal library.

Deep-extraction example

Suppose the first odd extraction is $G_1(q)$, the odd extraction of G_1 is $G_2(q)$, and the odd extraction of G_2 is zero. Then the original series has $a(8n+7)=0$ identically. The derivation report records this as three applications of the 2-dissection operator, not as one unexplained leap.

For a general residue r modulo p^d , read the base- p digits from least significant to most significant. At level j , select the component indexed by the j -th digit. This order is important because the first extraction acts on the original index. The app displays the digit path and the corresponding progression after every stage. When an exact stage fails, the remaining coefficient extraction is still shown, but the proof trace ends at the last justified symbolic identity.

5 5. Eta quotients, characters, and Sturm proofs

Eta conversion

Because

$$f_m = q^{-m/24} \eta(mz),$$

an f-product has a q-shift that must be tracked. For an eta quotient product $\eta(\delta z)^{r_\delta}$, the candidate weight is one half of the exponent sum.

Modularity checks

At level N, the app verifies that every delta divides N and checks

$$\sum_{\delta|N} \delta r_\delta \equiv 0 \pmod{24}, \quad \sum_{\delta|N} \frac{N}{\delta} r_\delta \equiv 0 \pmod{24}.$$

It then computes every cusp order using Ligozat's formula. Nonnegative orders give holomorphy; positive orders give cuspidality.

Character

For an integral-weight eta quotient, the app displays the encoded quadratic character kernel D, corresponding to a Kronecker-symbol character. Two forms used in a Sturm congruence must have the same weight and character.

Sturm certificate

The certificate module checks the hypotheses on both sides, confirms equal q-shifts, computes

$$B = \left\lfloor \frac{k}{12} [SL_2(\mathbb{Z}) : \Gamma_0(N)] \right\rfloor,$$

and tests coefficients through B. Only then does it display Certified.

Limitation

A progression $a(pn+r)$ is not automatically a modular form on the original group. The app will not infer a Sturm proof for a progression merely from a finite extraction. Supply a proven modular representation of the extracted series first.

Choosing a level

For an eta quotient, begin with the least common multiple of all eta subscripts, but do not assume it is automatically the optimal modular level. The Newman congruences may fail at that level

and hold after multiplication by an auxiliary eta quotient or after increasing the level. The Eta-Multiplier Lab searches for pole-clearing and congruence-correcting factors, while displaying the resulting weight and Sturm cost.

A small Sturm bound is convenient but not the only objective. Prefer a multiplier that preserves the intended arithmetic progression and yields a character compatible with the comparison form. The app's search is combinatorial within chosen exponent bounds; it does not prove that no multiplier exists outside those bounds. Record the search range whenever a negative result is used to guide further work.

6 6. Congruence mining and infinite families

Finite searches

The Congruence Miner tests whether $a(kn+r)$ is divisible by M through a chosen coefficient limit. Full sweeps examine all residues for a fixed stride. Hunter mode searches across strides and removes obvious subprogressions of earlier hits.

Heatmaps

The Visualization Studio displays the percentage of tested coefficients divisible by M for many pairs (k,r) . A cell at 100 percent is a useful signal but remains finite evidence.

Infinite-family detection

The Infinite Family Miner looks for progressions whose steps and offsets fit geometric formulas. This pattern detection can suggest Tang-type or p -adic families. It does not prove induction in the exponent. A proof usually requires an operator matrix, modular equation, or recurrence.

Recommended workflow

First generate enough terms to avoid accidental short patterns. Second filter trivial refinements. Third identify a plausible operator or dissection mechanism. Fourth prove the base case and recurrence. Finally use the app's LaTeX report to document the computational discovery separately from the proof.

Example

If a function appears to satisfy $a(5^\alpha n + \text{lambda_alpha})$ congruent to 0 modulo 5^α , use the miner to estimate lambda_alpha , then apply U_5 repeatedly in the Operator Sandbox and search for a stable finite-dimensional matrix action.

Turning a pattern into a theorem

After the miner finds a progression, verify it at a substantially larger limit than the discovery limit. Examine neighboring residues to understand whether the result is isolated or part of a larger dissection. Apply U_p or repeated extraction to search for a recurrence. If a finite-dimensional space appears, use the Relation Finder to identify a basis and compute the operator matrix.

For an infinite family modulo p^α , the key step is usually an induction mechanism: a matrix whose entries gain p -adic valuation, a modular equation, or an explicit recurrence between extracted generating functions. The app can discover the likely matrix entries and test valuations, but the final proof should state why the recurrence holds for every α and why the initial vector has the required divisibility.

7 7. Hecke operators, theta, rank, and crank

U and V operators

For $F = \sum a(n)q^n$,

$$F \mid U_p = \sum a(pn)q^n, \quad F \mid V_p = F(q^p).$$

These operators are useful for progression extraction and oldform constructions.

Integral-weight Hecke operator

For p not dividing the level,

$$b(n) = a(pn) + \chi(p)p^{k-1}a(n/p).$$

The app lets you enter k and $\chi(p)$. Terms with nonintegral n/p are treated as zero.

Half-integral convention

The $T(p^2)$ panel implements the displayed convention with an effective middle multiplier ϵ_p . Because authors normalize the character factor differently, copy the formula from the panel and verify that it matches the convention in your source before using it in a paper.

Theta operator

$\Theta = q \frac{d}{dq}$ sends $a(n)$ to $n a(n)$. Iterating it produces $n^r a(n)$, useful for weighted partition statistics and modular congruences.

Rank and crank moments

The Moment Lab expands the standard two-variable rank and crank generating functions and computes even moments. Select Crank minus rank to explore smallest-parts-type differences. The current implementation is intended for moderate N because bivariate expansion is more expensive than ordinary q -series multiplication.

Operator conventions

Before using a Hecke output in a manuscript, write the operator definition explicitly. Normalizations differ between sources, particularly in half-integral weight. The sandbox therefore displays the exact coefficient law used. Compare it with the theorem you intend to cite, including the placement of the character and the power of p .

A practical eigenform test is to compute $T_p F$ for several primes not dividing the level, then use the Relation Finder to compare $T_p F$ with F . If the space is one-dimensional and F is normalized,

proportionality strongly suggests the eigenvalue $a(p)$; a rigorous statement still depends on the modular-form membership and the dimension calculation. The Sage bridge can supply the dimension when available.

8 8. Relations, OEIS, and visual exploration

Exact and LLL relations

Enter several series of the same expected modular type. The app forms a coefficient matrix. Rational nullspace gives exact relations at the selected truncation; LLL searches for short integer combinations. Increase the precision to reject accidental relations.

A useful application is basis recognition. If a modular-form space has known dimension d and you enter more than d candidate eta quotients, the relation finder can reveal dependencies.

OEIS

The OEIS module sends only the integer coefficient list, not the formula. A match can suggest a known combinatorial interpretation or reference. Initial terms can be shifted or normalized before searching by editing the displayed list in a future extension; currently the search uses the coefficients beginning at $n=0$.

Coefficient plots

The animated plot shows growth and sign changes. The p-adic plot shows $\text{ord}_p(a(n))$; zero coefficients are omitted because their valuation is infinite. The heatmap is effective for spotting residue classes.

Caution

OEIS matches and LLL relations are discovery tools. Confirm index shifts, signs, normalizations, and enough additional coefficients before drawing mathematical conclusions.

Interpreting matches

An OEIS match may begin at a different offset, omit an initial zero, or use a sign convention. Read the entry carefully and compare more terms. Similarly, an LLL relation may be numerically short because the selected coefficient window is too small. Increase the degree and verify the relation using exact arithmetic.

Visualizations are most useful for questions that are difficult to see in a raw list: sudden growth, systematic zeros, periodic valuation spikes, or families of residue classes. Use the heatmap to formulate a precise candidate, then return to the Congruence Miner for the exact failing indices. Export plots for exploration, but base the proof on symbolic identities or modular certification rather than on visual appearance.

9 9. Saving, sharing, PDF, jobs, API, and Sage

Personal identity vault

Create a username and password, verify an identity, and save it with source notes. Passwords are stored as salted PBKDF2 hashes. On ephemeral free hosting, the SQLite file can disappear after a restart, so export JSON frequently. A persistent disk is recommended for a public multiuser deployment.

Share links

The Share Link Studio compresses the formula and selected options into a URL query parameter. Opening the link restores the corresponding module and formula without creating an account.

PDF reports

The Report Studio first looks for pdflatex or Tectonic. The Docker deployment installs a LaTeX engine. If no engine is available, the app produces a fallback PDF containing the LaTeX source.

Background jobs

Large expansions can run in a thread pool. Jobs are tied to the current browser session and server process; they are not a durable distributed queue. For production-scale computation, replace this layer with Redis and a worker system.

REST API

The included FastAPI service exposes health, expansion, dissection, and congruence-check endpoints. Interactive documentation is available at `/docs`.

Sage bridge

The optional Sage service computes modular-form dimensions and Sturm bounds using SageMath. Set `SAGE_BRIDGE_URL` to its address. The main app remains functional when the bridge is offline.

Deployment choices

Streamlit Community Cloud is the simplest option for the main interface, but its filesystem is ephemeral and system-level LaTeX availability can vary. Docker Compose gives the complete experience: web app, REST API, TeX engine, and a persistent data volume. The Sage bridge is optional because its image is much larger.

For a public installation, set memory and coefficient limits conservatively. Background threads share the same process and memory. Use a reverse proxy with HTTPS, keep dependencies updated, and

avoid placing secrets in the repository. The identity vault is a lightweight research convenience, not an institutional authentication system. For a multiuser production service, connect it to a managed database and established login provider.

10 10. Complete worked workflow and troubleshooting

Worked workflow

Consider

$$F(q) = \frac{1}{8} \left(\frac{f_2}{f_1^2} - \frac{f_1^6}{f_2^3} \right).$$

First enter the expression in the Automatic p-Dissection Solver with $p=3$. Export the exact derivation if available. Next open the Composite Lab and inspect each residue. If a component appears to vanish modulo a prime, run a higher coefficient check and visualize its p-adic valuations. If you obtain a simple eta quotient for the component, verify its level, weight, character, and cusp orders. Construct a same-space modular congruence and use the Sturm Certificate. Finally save the proven identity in the personal vault and generate a PDF report.

Common errors

unsupported operand type usually indicates an expression was sent to a pure eta-product parser instead of the linear-combination parser; the current dissection pages use the correct parser. **Series is not invertible** means the denominator has zero constant term. A failed Newman condition means the proposed eta quotient is not certified on that level by the implemented theorem. A blank OEIS or Sage result usually means the external service is unavailable.

Deployment

For Streamlit Community Cloud, upload `app.py`, `requirements.txt`, `packages.txt`, and the `.streamlit` directory. For the complete app plus REST API and LaTeX engine, use Docker Compose. For persistent identities, mount a volume at `RAMANUJAN_DATA_DIR`.

Research discipline

Keep generated conjectures, computational verifications, and proofs in separate sections of your notes. Record the coefficient limit and software version for every experiment. Re-run important identities at higher precision before submission. The app's main purpose is not only speed, but transparent reproducibility.

Final reproducibility checklist

A complete computational record should contain the input formula, notation definitions, coefficient limit, modulus, progression, identities used, source references, software version, and date. For a Sturm proof, include the level, weight, character, cusp orders, index, and exact bound. For a dissection proof, include the scaled identities and coefficient-extraction rule.

When troubleshooting deployment, first run `python -m py_compile app.py`, then start Streamlit locally, then test the health endpoint. For API problems, open `/docs` and submit a small expansion.

For Sage problems, test `/health` and confirm that the service is running under Sage's Python rather than ordinary Python. These checks isolate syntax, dependency, web, and mathematical-service failures in a systematic order.